

Your central idea is correct: **virtual threads target higher throughput for workloads with many blocking operations, not faster execution of an individual request**. Below is a corrected mental model, including creation APIs, carrier threads, blocking behavior, pooling, and practical limitations.

Virtual Threads vs Platform Threads

Java has two kinds of Thread:

```
1 Thread
2  └─ Platform thread
3  └─ Virtual thread
```

Both are instances of:

```
1 java.lang.Thread
```

The difference is primarily **who schedules them and how they use operating-system threads**.

1. Platform thread

A platform thread is the traditional Java thread.

```
1 Thread thread = new Thread(() -> {
2     System.out.println("Running");
3 });
4
5 thread.start();
```

Modern builder syntax:

```
1 Thread thread = Thread.ofPlatform()
2     .name("platform-worker")
3     .start(() -> {
4         System.out.println("Running");
5     });
```

A platform thread is implemented as a thin wrapper around an operating-system thread. While the Java platform thread is alive, it captures its corresponding OS thread.

[\[docs.oracle.com\]](https://docs.oracle.com)

Conceptually:

```
1 Java platform thread 1 → OS thread 1
2 Java platform thread 2 → OS thread 2
```

```
3 Java platform thread 3 → OS thread 3
```

This is effectively a:

```
1 1 Java platform thread : 1 OS thread
```

relationship.

2. Cost of platform threads

Platform threads are relatively expensive because each thread requires resources such as:

- An operating-system thread
- Native stack memory
- JVM bookkeeping
- OS scheduling
- Context switching

This does not mean that starting one platform thread is always extremely slow. It means that creating and maintaining a **very large number** of platform threads becomes expensive.

For example:

```
1 100 platform threads      → usually manageable
2 1,000 platform threads    → potentially expensive
3 100,000 platform threads → generally impractical
```

This is one reason traditional Java applications use thread pools:

```
1 ExecutorService executor =
2   Executors.newFixedThreadPool(100);
```

Instead of creating a platform thread for every task, the application creates a limited number and reuses them:

```
1 Task 1 → Worker 1
2 Task 2 → Worker 2
3 Task 3 → Worker 1 reused
4 Task 4 → Worker 2 reused
```

Thread pools help control:

- Native thread creation
- Memory consumption
- OS scheduling overhead
- Maximum concurrency
- Downstream resource pressure

3. Platform threads and blocking I/O

Suppose a platform thread performs a database query:

```
1 String result = database.query();
```

The thread may spend most of its time waiting:

```
1 Send database query
2     ↓
3 Wait 500 milliseconds
4     ↓
5 Receive response
6     ↓
7 Continue processing
```

During that waiting period:

- The platform thread is not doing useful application computation
- Its corresponding OS thread remains occupied
- That OS thread cannot run another Java platform thread

Conceptually:

```
1 Platform thread
2     ↓
3 OS thread
4     ↓
5 Waiting for database response
```

The CPU is not necessarily busy, but the OS thread remains unavailable to execute another Java task.

This limits throughput when an application has thousands of concurrent blocking operations.

4. Virtual threads

A virtual thread is a lightweight Thread managed and scheduled by the JDK rather than being permanently tied to one OS thread. The JVM can map a large number of virtual threads onto a much smaller number of platform threads. openjdk.org, docs.oracle.com

Conceptually:

```
1 Virtual thread 1 ┌
2 Virtual thread 2 ┤
3 Virtual thread 3 ┤ — Carrier platform threads — OS threads
4 Virtual thread 4 ┤
5 Virtual thread 5 └
```

This is known as an M:N relationship:

```
1 M virtual threads : N platform carrier threads
```

A virtual thread is still a real Java Thread:

```
1 Thread.currentThread();
```

It has:

- An identity
- A stack trace
- Interruption support
- Thread-local support
- Debugging and observability support

However, it is not permanently associated with one OS thread.

5. Introduction timeline

Virtual threads have an important version history:

```
1 JDK 19 → First preview
2 JDK 20 → Second preview
3 JDK 21 → Finalized as a standard feature
```

Therefore, your statement:

Virtual threads were introduced in JDK 19

is correct if you mean **introduced as a preview feature**.

For normal production use without preview flags, the key version is:

```
1 JDK 21
```

JEP 444 finalized virtual threads in Java 21 after preview releases in Java 19 and Java 20.

[\[openjdk.org\]](https://openjdk.org)

6. Carrier threads

The platform thread on which a virtual thread is currently executing is called its **carrier thread**.

```
1 Virtual thread
2     ↓ mounted onto
3 Carrier platform thread
4     ↓ backed by
5 OS thread
```

Suppose the JVM scheduler has four carrier threads:

```
1 Carrier 1
2 Carrier 2
3 Carrier 3
4 Carrier 4
```

It may schedule thousands of virtual threads across those carrier threads:

```
1 VT-1    → Carrier 1
2 VT-2    → Carrier 2
3 VT-3    → Carrier 3
4 VT-4    → Carrier 4
5
6 VT-5    → waiting
7 VT-6    → waiting
8 VT-7    → waiting
9 ...
```

The JVM's virtual-thread scheduler assigns runnable virtual threads to carrier platform threads. The operating system still schedules those carrier platform threads onto processor cores. openjdk.org

Important correction

The JVM does not normally create a new carrier thread for every virtual thread.

Also, it is not best to say:

The JVM keeps increasing OS threads according to how many virtual-thread objects exist.

The normal model is:

```
1 Many virtual threads
2     ↓
3 Scheduled over a relatively small carrier pool
```

The carrier pool's target parallelism is generally related to available processors, not directly to the total number of virtual threads.

7. Mounting a virtual thread

When a virtual thread is ready to execute, the scheduler mounts it onto an available carrier thread.

```
1 Virtual thread is runnable
2     ↓
3 Scheduler selects a carrier
4     ↓
5 Virtual thread is mounted
6     ↓
7 Carrier executes its Java code
```

Example:

```
1 Thread.startVirtualThread(() -> {
2     performCalculation();
3 });
```

During `performCalculation()`, the virtual thread is mounted on a carrier platform thread.

A virtual thread can only execute Java instructions while it is mounted to a carrier.

8. Unmounting during blocking operations

Suppose the virtual thread performs a supported blocking operation:

```
1 String response = httpClient.send(request);
```

Conceptually:

```
1 Virtual thread executing
2     ↓
3 Starts blocking I/O
4     ↓
5 Virtual thread is suspended and unmounted
6     ↓
7 Carrier thread is released
8     ↓
9 Carrier executes another virtual thread
```

When the I/O operation can continue:

```
1 I/O response becomes available
2     ↓
3 Virtual thread becomes runnable
4     ↓
5 Scheduler mounts it onto an available carrier
6     ↓
7 Execution resumes
```

The virtual thread may resume on a **different carrier thread**. It has no permanent attachment to the original carrier. Virtual threads can unmount during blocking operations such as I/O, allowing the carrier to execute other virtual threads. [\[docs.oracle.com\]](https://docs.oracle.com), [\[openjdk.org\]](https://openjdk.org)

Example:

```
1 VT-1 begins on Carrier-1
2 VT-1 waits for database I/O
3 VT-1 unmounts
4
5 Carrier-1 executes VT-2
6
7 Database result arrives
8 VT-1 resumes on Carrier-3
```

From the application programmer's perspective, the code remains sequential:

```
1 User user = loadUser();
2 List<Order> orders = loadOrders(user);
3 createResponse(user, orders);
```

You do not normally manage mounting and unmounting manually.

9. Virtual threads improve throughput, not latency

Your statement is correct:

The principal goal of virtual threads is throughput, not lower latency.

Throughput means:

```
1 How many tasks or requests the application completes
2 during a particular period
```

Latency means:

```
1 How long one particular task or request takes
```

Suppose a downstream API takes one second to respond.

Using a platform thread:

```
1 One request latency ≈ 1 second
```

Using a virtual thread:

```
1 One request latency ≈ 1 second
```

The virtual thread does not make the external API respond faster.

The improvement is that the application can wait for many such responses concurrently without requiring one dedicated OS thread for each operation:

```
1 Platform threads:
2 200 blocked requests → approximately 200 occupied OS threads
3
4 Virtual threads:
5 20,000 blocked requests → multiplexed over a smaller carrier pool
```

Virtual threads are intended primarily for high-throughput applications containing many concurrent tasks that spend significant time waiting for I/O. They are not intended to accelerate long-running CPU-intensive computation. openjdk.org, docs.oracle.com

10. Creating a virtual thread

Thread.ofVirtual()

Create and start a named virtual thread:

```
1 Thread thread = Thread.ofVirtual()
2     .name("payment-request")
3     .start(() -> {
4         processPayment();
5     });
```

The start() method both creates and begins execution.

You can also create an unstarted thread:

```
1 Thread thread = Thread.ofVirtual()
2     .name("payment-request")
3     .unstarted(() -> {
4         processPayment();
```



```
5     });  
6  
7     thread.start();
```

Thread.startVirtualThread()

A concise alternative is:

```
1 Thread thread = Thread.startVirtualThread(() -> {  
2     processPayment();  
3 });
```

This creates and starts a virtual thread immediately.

11. Virtual-thread-per-task executor

The correct method name is:

```
1 Executors.newVirtualThreadPerTaskExecutor()
```

Not:

```
1 Executors.newVirtualThreadTaskPerExecutor()
```

Example:

```
1 try (ExecutorService executor =  
2     Executors.newVirtualThreadPerTaskExecutor()) {  
3  
4     Future<String> future = executor.submit(() -> {  
5         return callExternalService();  
6     });  
7  
8     System.out.println(future.get());  
9 }
```

This executor creates a **new virtual thread for every submitted task**:

```
1 Task 1 → Virtual thread 1  
2 Task 2 → Virtual thread 2  
3 Task 3 → Virtual thread 3  
4 Task 4 → Virtual thread 4
```

It does not maintain a small reusable pool of virtual threads.

That is intentional because virtual threads are designed to be inexpensive and short-lived.

12. Do not pool virtual threads

With platform threads, pooling is necessary because platform threads are expensive:

```
1 Executors.newFixedThreadPool(100);
```

With virtual threads, the recommended model is typically:

```
1 One virtual thread per task
```

Use:

```
1 Executors.newVirtualThreadPerTaskExecutor();
```

Do not normally create:

```
1 Executors.newFixedThreadPool(100);
```

with virtual threads as reusable workers.

A virtual thread should generally represent the lifetime of one task:

```
1 Task begins
2   ↓
3 Virtual thread created
4   ↓
5 Task executes and blocks as needed
6   ↓
7 Task completes
8   ↓
9 Virtual thread terminates
```

The scheduler already multiplexes the virtual threads over carrier threads. Creating an application-level virtual-thread pool usually adds unnecessary queueing and removes the thread-per-task benefit.

13. Complete I/O-oriented example

```
1 import java.util.ArrayList;
2 import java.util.List;
3 import java.util.concurrent.ExecutorService;
4 import java.util.concurrent.Executors;
5 import java.util.concurrent.Future;
6
7 public class VirtualThreadExample {
```

```

8
9     public static void main(String[] args)
10         throws Exception {
11
12         try (ExecutorService executor =
13             Executors.newVirtualThreadPerTaskExecutor())
14         {
15             List<Future<String>> futures =
16                 new ArrayList<>();
17
18             for (int requestId = 1;
19                 requestId <= 10_000;
20                 requestId++) {
21
22                 int currentRequestId = requestId;
23
24                 futures.add(
25                     executor.submit(() ->
26                         processRequest(currentRequestId)
27                     )
28                 );
29             }
30
31             for (Future<String> future : futures) {
32                 System.out.println(future.get());
33             }
34         }
35     }
36
37     private static String processRequest(int requestId)
38         throws InterruptedException {
39
40         Thread.sleep(1_000);
41
42         return "Completed request " + requestId
43             + " on " + Thread.currentThread();
44     }
45 }

```

Thread.sleep() suspends the virtual thread instead of forcing its carrier to remain occupied for the entire sleep period.

The carrier can execute other runnable virtual threads while that virtual thread waits.

14. Virtual threads do not increase CPU parallelism

Suppose the machine has eight CPU cores and you create one million CPU-intensive virtual threads:

```
1  for (int index = 0; index < 1_000_000; index++) {  
2      Thread.startVirtualThread(() -> {  
3          performHeavyCpuCalculation();  
4      });  
5  }
```

This does not give you one million computations executing simultaneously.

Actual CPU parallelism is still physically limited:

```
1  8 CPU cores  
2      ↓  
3  Approximately 8 computations can make real CPU progress  
4  at any instant
```

The other tasks wait to be scheduled.

For CPU-bound work, a bounded executor or parallel computation framework is generally more suitable:

```
1  ExecutorService cpuExecutor =  
2      Executors.newFixedThreadPool(  
3          Runtime.getRuntime().availableProcessors()  
4      );
```

Virtual threads are most beneficial when tasks spend much of their lifetime waiting rather than calculating. docs.oracle.com

15. Do not use thread pools to limit downstream concurrency

Previously, developers sometimes used pool size as both:

1. A thread-management mechanism
2. A concurrency limiter

For example:

```
1 ExecutorService executor =  
2     Executors.newFixedThreadPool(20);
```

This indirectly limits database calls to 20.

A virtual-thread-per-task executor does not provide that same bound:

```
1 Executors.newVirtualThreadPerTaskExecutor();
```

If 100,000 tasks are submitted, a very large number of virtual threads may concurrently attempt to access:

- A database
- An HTTP service
- A file system
- A limited connection pool
- A rate-limited API

Virtual threads make threads abundant. They do not make downstream resources abundant.

Use an explicit concurrency limiter such as a semaphore:

```
1 private static final Semaphore DATABASE_LIMIT =  
2     new Semaphore(20);  
3  
4 static String callDatabase() throws InterruptedException {  
5     DATABASE_LIMIT.acquire();  
6  
7     try {  
8         return executeDatabaseQuery();  
9     } finally {  
10        DATABASE_LIMIT.release();  
11    }  
12 }
```

This separates the concerns:

```
1 Virtual threads  
2     → Represent concurrent tasks  
3  
4 Semaphore or connection pool  
5     → Protects a limited external resource
```

16. Virtual threads and ThreadLocal

Virtual threads support ThreadLocal variables, but care is required because an application may create an extremely large number of virtual threads. JDK 21 finalized virtual threads with guaranteed support for thread-local variables. [\[openjdk.org\]](https://openjdk.org), [\[docs.oracle.com\]](https://docs.oracle.com)

For example:

```
1 private static final ThreadLocal<RequestContext> CONTEXT =  
2     new ThreadLocal<>();
```

Each virtual thread can receive a separate value:

```
1 Thread.startVirtualThread(() -> {  
2     CONTEXT.set(new RequestContext());  
3  
4     try {  
5         processRequest();  
6     } finally {  
7         CONTEXT.remove();  
8     }  
9 });
```

The value follows the **virtual thread**, not the carrier thread.

Therefore, even if a virtual thread resumes on another carrier:

```
1 VT-1 on Carrier-1 → CONTEXT = A  
2 VT-1 unmounts  
3 VT-1 resumes on Carrier-3 → CONTEXT is still A
```

However, avoid using ThreadLocal as a per-thread cache when creating hundreds of thousands of virtual threads. A separate cached object for every virtual thread can consume substantial memory.

17. Pinning and JDK versions

In older virtual-thread implementations, certain blocking operations could cause a virtual thread to remain attached to its carrier. This is known as **pinning**.

Conceptually:

```
1 Normal blocking:  
2 Virtual thread blocks  
3     ↓  
4 Unmounts
```

```
5      ↓
6  Carrier is reusable
1  Pinned blocking:
2  Virtual thread blocks
3      ↓
4  Remains mounted
5      ↓
6  Carrier is also blocked
```

JDK 24 delivered JEP 491, which allows virtual threads blocked inside synchronized methods and statements to release their carrier threads, eliminating nearly all monitor-related pinning cases. [\[openjdk.org\]](https://openjdk.org)

This means advice commonly written for JDK 21, such as always replacing synchronized with ReentrantLock solely to avoid virtual-thread pinning, is no longer generally applicable when running on JDK 24 or later.

Some uncommon cases, particularly certain native or foreign-function interactions, can still prevent unmounting. The practical lesson is to test and profile the actual workload rather than assuming every blocking operation will scale perfectly.

18. Platform threads versus virtual threads

Platform threads

```
1  Managed primarily through:
2  JVM and OS thread scheduling
3
4  Mapping:
5  Approximately 1 Java platform thread to 1 OS thread
6
7  Creation cost:
8  Relatively expensive
9
10 Memory:
11 Relatively large native stack and OS resources
12
13 Blocking:
14 Occupies the underlying OS thread
15
16 Pooling:
```

- 17 Common and usually necessary
- 18
- 19 Best suited for:
- 20 CPU-oriented workers
- 21 Small, controlled concurrency
- 22 Long-lived infrastructure threads

Virtual threads

- 1 Managed primarily through:
- 2 JDK virtual-thread scheduler
- 3
- 4 Mapping:
- 5 Many virtual threads to fewer carrier platform threads
- 6
- 7 Creation cost:
- 8 Relatively inexpensive
- 9
- 10 Memory:
- 11 Smaller, dynamically managed stack representation
- 12
- 13 Blocking:
- 14 Usually unmounts and releases the carrier
- 15
- 16 Pooling:
- 17 Normally unnecessary
- 18
- 19 Best suited for:
- 20 Large numbers of blocking I/O tasks
- 21 Thread-per-request server applications
- 22 Simple synchronous request-processing code

19. Corrected mental model

Your original model can be refined to:

- 1 Platform thread:
- 2 Java Thread
- 3 ↓ permanently associated for its lifetime
- 4 OS thread
- 5 ↓

6 Scheduled by operating system

1 Virtual thread:
2 Java Thread managed by JDK
3 ↓ temporarily mounted
4 Carrier platform thread
5 ↓
6 OS thread

When a virtual thread reaches supported blocking I/O:

1 Virtual thread blocks
2 ↓
3 Virtual thread is suspended and unmounted
4 ↓
5 Carrier is released
6 ↓
7 Another virtual thread uses the carrier

When the operation becomes ready:

1 Virtual thread becomes runnable
2 ↓
3 Scheduler selects an available carrier
4 ↓
5 Virtual thread resumes

The selected carrier does not need to be the same carrier used previously.

20. Final corrections

Version

Instead of:

Virtual threads were introduced in JDK 19.

Use:

Virtual threads were previewed in JDK 19 and JDK 20, then finalized as a standard feature in JDK 21.

Executor method

Instead of:

1 Executors. newVirtualThreadTaskPerExecutor() ;
--

Use:

```
1 Executors.newVirtualThreadPerTaskExecutor();
```

Carrier management

Instead of:

The JVM increases OS threads according to how many virtual threads were created.

Use:

The JDK schedules many virtual threads over a smaller pool of carrier platform threads.
The total virtual-thread count does not directly determine the number of OS threads.

Blocking behavior

Instead of:

Every waiting virtual thread always detaches from its OS thread.

Use:

For supported blocking operations, the virtual thread normally unmounts from its carrier, allowing the carrier to execute other virtual threads.

Performance objective

Your statement is accurate:

```
1 Virtual threads primarily improve throughput and scalability
2 for high-concurrency blocking workloads.
3
4 They do not inherently reduce the latency of one operation.
```

Main improvements to your notes

- Distinguished **JDK-managed virtual threads**, **carrier platform threads**, and **OS threads**.
- Corrected the virtual-thread executor method name.
- Clarified that virtual threads were previewed in JDK 19 and finalized in JDK 21.
- Explained that carrier thread count does not grow one-for-one with virtual-thread count.
- Separated throughput, latency, concurrency, and CPU parallelism.
- Added the production rule that virtual threads should not replace explicit limits on database or downstream-service concurrency.